

Fiche 320 — Interfaces avec C et Fortran : `.C`, `.Call`, `SEXP` et `PROTECT`

Matière	Code · Langage R
Cours source	R Core Team, <i>Writing R Extensions</i> 4.6.1 — chapitre 5 « System and foreign language interfaces » (§5.1 accès au système, §5.2 <code>.C</code> et <code>.Fortran</code> , §5.3 <code>dyn.load</code> , §5.4 enregistrement des routines natives, §5.9 manipuler les objets R en C, §5.9.1 le ramasse-miettes, §5.9.3 les types, §5.10 <code>.Call</code> et <code>.External</code>)
Sources d'appoint	R Core Team, <i>R Internals</i> 4.6.1 — §1.1 « SEXPs », §1.1.1 « SEXPTYPEs »
Difficulté	Avancé — la frontière entre R et le code compilé
Temps d'étude estimé	1 h 30
Prérequis	Fiches 303, 317, 319 (types, profilage, structure d'un paquet)
Concepts clés	<code>.C</code> et <code>.Fortran</code> , la table de correspondance des types , <code>NAOK</code> , argument <code>PACKAGE</code> , <code>dyn.load</code> / <code>dyn.unload</code> , enregistrement des routines , <code>R_registerRoutines</code> , <code>R_init_dll</code> , <code>SEXP</code> et <code>SEXPREF</code> , <code>.Call</code> et <code>.External</code> , <code>PROTECT</code> / <code>UNPROTECT</code> , la pile de protection , <code>Rf_allocVector</code> , <code>Rf_coerceVector</code> , les <code>SEXPTYPE</code> , <code>Rf_error</code> , préfixe <code>Rf_</code>
À retenir en priorité	Le tableau des types · <code>.C</code> copie, <code>.Call</code> non · <code>PROTECT</code> protège l'objet, pas le pointeur · l'argument <code>PACKAGE</code> · regarder d'abord si R suffit.

Vue d'ensemble

QUATRE INTERFACES `.C` `.Fortran` -> vecteurs atomiques, arguments COPIES
 `.Call` `.External` -> objets R natifs (SEXP), PAS DE COPIE

LE CONSEIL PREALABLE

« Before you decide to use `.Call` or `.External`, YOU SHOULD LOOK AT OTHER ALTERNATIVES. First, consider working in interpreted R code ; IF THIS IS FAST ENOUGH, THIS IS NORMALLY THE BEST OPTION. »

TABLE DES TYPES (.C / .Fortran)

R storage mode	C type	Fortran type
logical	int *	INTEGER
integer	int *	INTEGER
double	double *	DOUBLE PRECISION
complex	Rcomplex *	DOUBLE COMPLEX
character	char **	CHARACTER(255)
raw	unsigned char *	-- aucun --

!! int et INTEGER sont 32 bits SUR TOUTES LES PLATEFORMES

REGLES DE .C jusqu'a 65 arguments • arguments COPIES a l'aller ET au retour
le retour se fait DANS LES ARGUMENTS -- la fonction C est void
NAOK = FALSE : NA, NaN, Inf, -Inf provoquent une ERREUR
!! PACKAGE = "... " -- « its use is HIGHLY DESIRABLE »

.Call SEXP f(SEXP a, SEXP b) jusqu'a 65 objets
.External SEXP f(SEXP args) UN SEUL argument, une pairlist
!! « NEITHER .Call NOR .External COPY THEIR ARGUMENTS : you should treat
arguments you receive through these interfaces AS READ-ONLY. »

LE RAMASSE-MIETTES

PROTECT(p) met l'OBJET a l'abri -- « it is THE OBJECT which is protected,
NOT THE POINTER VARIABLE »

UNPROTECT(n) depile les n derniers ; les appels DOIVENT S'EQUILIBRER
pile de taille FIXE (10 000 par défaut)

ENREGISTRER R_registerRoutines() dans R_init_<dll>() -> plus RAPIDE et VERIFIE

Le problème posé. « *Using C code to speed up the execution of an R function is often very fruitful. Traditionally this has been done via the .C function. However, if a user wants to write C code using internal R data structures, then that can be done using the .Call and .External functions.* » (§5.9)

Le conseil qui doit précéder tout le reste (§5.9). « *Before you decide to use `.call` or `.External`, you should look at other alternatives. First, consider working in interpreted R code ; if this is fast enough, this is normally the best option. You should also see if using `.C` is enough. If the task to be performed in C is simple enough involving only atomic vectors and requiring no call to R, `.C` suffices. ... These interfaces allow much more control, but they also impose much greater responsibilities so need to be used with care.* »

Trois marches, dans cet ordre : du R interprété — **normalement le meilleur choix** —, puis `.C`, puis `.call`. Et la fiche 317 ajoute l'étape zéro : **profiler d'abord**, pour savoir si le goulot est bien là où on le croit. Le §3.2 disait précisément que le profilage sert à « *identify ... pieces of R code that could benefit from being replaced by compiled code* ».

Une convention de nommage (§5). « *Many of the functions described here have entry-point names with a `Rf_` prefix : if they are called from C code (but not C++ code as from R 4.5.0) that prefix can be omitted. Users are encouraged to use the prefix when writing new C code.* »

● Concept 1 — Accès au système depuis C

Règle (§5.1). « *Access to operating system functions is via the R functions `system` and `system2`. The details will differ by platform, and about all that can safely be assumed is that the first argument will be a string command that will be passed for execution (not necessarily by a shell) and the second argument to `system` will be `internal` which if true will collect the output of the command into an R character vector.* »

« *On POSIX-compliant OSes these commands pass a command-line to a shell : Windows is not POSIX-compliant and there is a separate function `shell` to do so.* »

« *The function `system.time` is available for timing. Timing on child processes is only available on Unix-alikes, and may not be reliable there.* »

C'est la reprise du §14.3 (fiche 318), avec une précision : « **not necessarily by a shell** » — on ne peut rien supposer de plus que le passage de la chaîne.

● Concept 2 — `.C` et `.Fortran`

DÉFINITION (§5.2).

« These two functions provide an interface to compiled code that has been linked into R, either at build time or via `dyn.load`. They are primarily intended for compiled C and Fortran code respectively, but the `.C` function can be used with other languages which can generate C interfaces, for example C++ . »

« The first argument to each function is a character string specifying the symbol name as known to C or Fortran ... (That the symbol is loaded can be tested by, for example, `is.loaded("cg")` .) »

« There can be up to 65 further arguments giving R objects to be passed to compiled code. Normally these are copied before being passed in, and copied again to an R list object when the compiled code returns. If the arguments are given names, these are used as names for the components in the returned list object (but not passed to the compiled code). »

2.1 La table de correspondance des types

Règle (§5.2). « The following table gives the mapping between the modes of R atomic vectors and the types of arguments to a C function or Fortran subroutine. »

Mode de stockage R	Type C	Type Fortran
<code>logical</code>	<code>int *</code>	<code>INTEGER</code>
<code>integer</code>	<code>int *</code>	<code>INTEGER</code>
<code>double</code>	<code>double *</code>	<code>DOUBLE PRECISION</code>
<code>complex</code>	<code>Rcomplex *</code>	<code>DOUBLE COMPLEX</code>
<code>character</code>	<code>char **</code>	<code>CHARACTER(255)</code>
<code>raw</code>	<code>unsigned char *</code>	<code>aucun</code>

La mise en garde de portabilité (§5.2). « *On all R platforms `int` and `INTEGER` are 32-bit. Code ported from S-PLUS (which uses `Long *` for logical and integer) will not work on all 64-bit platforms (although it may appear to work on some, including `x86_64` Windows). Note also that if your compiled code is a mixture of C functions and Fortran subprograms the argument types must match as given in the table above.* »

« **It may appear to work on some** » est le pire des cas : le code semble correct, jusqu'à ce qu'il tourne ailleurs.

Sur `Rcomplex` (§5.2) : *« a structure with double members `r` and `i` defined in the header file `R_ext/Complex.h`. (On most platforms this is stored in a way compatible with the C99 `double complex` type : however, it may not be possible to pass `Rcomplex` to a C99 function ... Nor need it be compatible with a C++ `complex` type. Moreover, the compatibility can depend on the optimization level set for the compiler.) »*

Sur les chaînes et Fortran : « *Only a single character string of fixed length can be passed to or from Fortran (the length is not passed), and the success of this is compiler-dependent : its use was formally deprecated in 2019.* »

2.2 Les valeurs spéciales

Règle (§5.2). *« Logical values are sent as 0 (`FALSE`), 1 (`TRUE`) or `INT_MIN` = -2147483648 (`NA` , but only if `NAOK` is true), and the compiled code should return one of these three values. (Non-zero values other than `INT_MIN` are mapped to `TRUE` .) »*

« *Note that the use of `int *` for Fortran `logical` is not guaranteed to be portable ... it is better to pass integers and convert to/from Fortran `logical` in a Fortran wrapper.* »

Règle (§5.2). « *Unless formal argument `NAOK` is true, all the other arguments are checked for missing values `NA` and for the IEEE special values `NaN` , `Inf` and `-Inf` , and the presence of any of these generates an error. If it is true, these values are passed unchecked.* »

`NAOK` est donc un garde-fou activé par défaut. Le passer à `TRUE` transfère au code C la responsabilité de traiter les valeurs spéciales — ce qui est nécessaire dès qu'on veut y accéder, et dangereux si on ne les gère pas.

2.3 L'argument `PACKAGE`

Règle (§5.2). « *Argument `PACKAGE` confines the search for the symbol name to a specific shared object (or use `"base"` for code compiled into R). Its use is highly desirable, as there is no way to avoid two package writers using the same symbol name, and such name clashes are normally sufficient to cause R to crash.* »

« (If it is not present and the call is from the body of a function defined in a package namespace, the shared object loaded by the first (if any) `useDynLib` directive will be used.) »

« **Sufficient to cause R to crash** » — le mot est fort. Deux paquets définissant `convolve` dans leur DLL, sans `PACKAGE`, et l'un appelle le code de l'autre avec des arguments qui ne lui conviennent pas. Il n'y a **aucun système de noms** au niveau des symboles compilés — c'est ce que l'enregistrement (concept 4) vient corriger.

C'est aussi ce que vérifie `R CMD check`, point 10 de la fiche 319 : « *all foreign function calls ... are tested to see if they have a `PACKAGE` argument* ».

La contrainte de retour (§5.2). « *Note that the compiled code should not return anything except through its arguments : C functions should be of type `void` and Fortran subprograms should be subroutines.* »

C'est ce qui distingue `.C` de `.Call` : avec `.C`, le résultat sort par les arguments modifiés, que R recopie ensuite dans une liste. La fonction C **ne rend rien**.

● **Concept 3** — `.Call` et `.External`

DÉFINITION (§5.9).

« **A call to `.Call` is very similar to `.C`** » :

```
.Call("convolve2", a, b)
```

« *The first argument should be **a character string giving a C symbol name of code that has already been loaded** into R. **Up to 65 R objects can be passed as arguments**. The C side of the interface is* » :

```
#include <R.h>
#include <Rinternals.h>

SEXP convolve2(SEXP a, SEXP b)
...

```

« **A call to `.External` is almost identical** » :

```
.External("convolveE", a, b)
```

« **but the C side of the interface is different, having only one argument** » :

```
SEXP convolveE(SEXP args)
...

```

« Here `args` is a `LISTSXP`, a Lisp-style pairlist from which the arguments can be extracted. »

	<code>.Call</code>	<code>.External</code>
Côté C	un argument par objet	un seul argument — une pairlist
Nombre d'objets	jusqu'à 65	quelconque
Le manuel dit	« <i>simpler to use</i> »	« <i>a little more general</i> »

La règle absolue (§5.9). « *Neither `.Call` nor `.External` copy their arguments : you should treat arguments you receive through these interfaces as read-only.* »

C'est l'exception à la sémantique de copie de R (fiche 302), et elle est à la charge du programmeur. Modifier un argument reçu par `.Call`, c'est modifier l'objet de l'appelant — silencieusement, et souvent d'autres objets avec lui, R partageant les représentations.

● Concept 4 — `SEXP`, ou tout est un pointeur

DÉFINITION (§5.9).

« *All the R objects you will deal with will be handled with the type `SEXP`, which is a pointer to a structure with `typedef SEXPREF`. Think of this structure as a variant type that can handle all the usual types of R objects* — vectors of various modes, functions, environments, language objects and so on. »

« *Think rather of a model such as that used by Visual Basic, in which R objects are handed around in C code (as they are in interpreted R code) as the variant type, and the appropriate part is extracted for, for example, numerical calculations, only when it is needed. As in interpreted R code, much use is made of coercion to force the variant object to the right type.* »

(note 17 : « `SEXP` is an acronym for S imple EXP ressession, common in LISP-like language syntaxes. »)

COMPLÉMENT (R INTERNALS §1.1).

*« **What R users think of as variables or objects are symbols which are bound to a value.** The value can be thought of as either a **SEXP** (a pointer), or the structure it points to, a **SEXPREF** ... **So the basic building blocks of R objects are often called nodes.** »*

« *Note that the internal structure of the **SEXPREF** is not made available to R Extensions : rather **SEXP** is an opaque pointer, and the internals can only be accessed by the functions provided.* »

« Both types of node structure have as their first three fields a **64-bit** **sxpinfo** header and then three pointers (to the attributes and the previous and next node in a doubly-linked list) ... On a 32-bit platform a node occupies 32 bytes : on a 64-bit platform typically 56 bytes. »

« The first five bits of the **sxpinfo** header specify one of up to 32 **SEXPTYPE** s. »

4.1 Les **SEXPTYPE**

Règle (§5.9.3). « *The different R data types are represented in C by **SEXPTYPE** . Some of these are familiar from R and some are internal data types.* »

SEXPTYPE	Équivalent R
REALSXP	numeric , mode de stockage double
INTSXP	integer
CPLXSXP	complex
LGLSXP	logical
STRSXP	character
VECSXP	list (vecteur générique)
LISTSXP	pairlist
DOTSXP	un objet ...
NILSXP	NULL
SYMSXP	nom / symbole
CLOSXP	fonction ou fermeture
ENVSXP	environnement

*« Among the important **internal** SEXPTYPE s are `LANGSXP` , `CHARSXP` , `PROMSXP` , etc. (**although it is possible to return objects of internal types, it is unsafe to do so as assumptions are made about how they are handled which may be violated at user-level evaluation**). »*

CONTRÔLE — RECONNAÎTRE UNE VIEILLE CONNAISSANCE.

Cette table **est** celle de la fiche 303, vue du côté C. `VECSXP` pour une liste, `LISTSXP` pour une pairlist, `CLOXP` pour une fermeture, `PROMSXP` pour une promesse — ce sont exactement les valeurs que rend `typeof()`.

Et *R Internals* §1.1.1 donne les numéros : `NILSXP` 0, `SYMSXP` 1, `LISTSXP` 2, `CLOXP` 3, `ENVSXP` 4, `PROMSXP` 5, `LANGSXP` 6, ... `LGLSXP` 10, puis 13 `INTSXP`, `REALSXP` 14, `CPLXSXP` 15, `STRSXP` 16, ... `VECSXP` 19, `EXPRSXP` 20, `BCODESXP` 21, `EXTPTRSXP` 22, `WEAKREFSXP` 23, `RAWSXP` 24, `OBJSXP` 25.

Le trou en 11-12 : « *Values 11 and 12 were used for internal factors and ordered factors and have since been withdrawn. Note that the `SEXPTYPE` numbers are stored in saved objects and that the ordering of the types is used, so the gap cannot easily be reused.* »
— un vestige gravé dans le format des fichiers `.RData`.

4.2 Vérifier et convertir

MÉTHODE (§5.9.3).

« Unless you are very sure about the type of the arguments, the code should check the data types. You can use functions like `Rf_isReal`, `Rf_isInteger` and `Rf_isString` ... Other such functions include `Rf_isNull`, `Rf_isSymbol`, `Rf_isLogical`, `Rf_isComplex`, `Rf_isExpression`, and `Rf_isEnvironment`. All of these take a `SEXP` as argument and return 1 or 0. »

« *What happens if the `SEXP` is not of the correct type ? Sometimes you have no other option except to generate an error. You can use the function `Rf_error` for this. It is usually better to coerce the object to the correct type.* »

```
newSexp = PROTECT(Rf_coerceVector(oldSexp, REALSXP));
```

« **Protection is needed as a new object is created** ; the object formerly pointed to by the `SEXP` is still protected but now unused. »

« **All the coercion functions do their own error-checking, and generate `NA`s with a warning or stop with an error as appropriate.** »

● Concept 5 — Le ramasse-miettes et `PROTECT`

DÉFINITION (§5.9.1).

« *We need to know a little about the way R handles memory allocation. The memory allocated for R objects is not freed by the user ; instead, the memory is from time to time garbage collected.* »

La règle centrale (§5.9.1). « *If you create an R object in your C code, you must tell R that you are using the object by using the `PROTECT` macro on a pointer to the object. This tells R that the object is in use so it is not destroyed during garbage collection.* »

« *Notice that it is the object which is protected, not the pointer variable. It is a common mistake to believe that if you invoked `PROTECT(p)` at some point then `p` is protected from then on, but that is not true once a new object is assigned to `p`.* »

« *Protecting an R object automatically protects all the R objects pointed to in the corresponding `SEXPREC` — for example all elements of a protected list are automatically protected.* »

La distinction objet / pointeur est l'erreur la plus fréquente, et le manuel la nomme comme telle.

`PROTECT(p)` protège **ce que `p` désigne à cet instant** ; réassigner `p` laisse le nouvel objet **sans protection**.

Règle — l'équilibre (§5.9.1). « *The programmer is solely responsible for housekeeping the calls to `PROTECT`. There is a corresponding macro `UNPROTECT` that takes as argument an `int` giving the number of objects to unprotect. The protection mechanism is stack-based, so `UNPROTECT(n)` unprotects the last `n` objects which were protected. The calls to `PROTECT` and `UNPROTECT` must balance when the user's code returns and should balance in all functions. R will warn about "stack imbalance in `.Call`" if the housekeeping is wrong.* »

L'exemple du manuel :

```
#include <R.h>
#include <Rinternals.h>
```

```
SEXP ab;
....
ab = PROTECT(Rf_allocVector(REALSXP, 2));
REAL(ab)[0] = 123.45;
REAL(ab)[1] = 67.89;
UNPROTECT(1);
```

POURQUOI PROTÉGER, ALORS QUE « C'EST JUSTE NOTRE CODE C QUI TOURNE » (§5.9.1).

« Now, the reader may ask **how the R object could possibly get removed during those manipulations**, as it is just our C code that is running. **As it happens, we can do without the protection in this example, but in general we do not know (nor want to know) what is hiding behind the R macros and functions we use, and any of them might cause memory to be allocated, hence garbage collection and hence our object `ab` to be removed. It is usually wise to err on the side of caution and assume that any of the R macros and functions might remove the object.** »

C'est un raisonnement de discipline, pas de nécessité locale : on protège parce qu'on **ne peut pas savoir** ce qu'une macro déclenche.

La pile a une taille finie (§5.9.1). « In some cases it is necessary to keep better track of whether protection is really needed. Be particularly aware of situations where a large number of objects are generated. The pointer protection stack has a fixed size (default 10,000) and can become full. It is not a good idea then to just `PROTECT` everything in sight and `UNPROTECT` several thousand objects at the end. It will almost invariably be possible to either assign the objects as part of another object (which automatically protects them) or unprotect them immediately after use. »

Les deux issues sont données : ranger l'objet dans un autre — la protection est alors héritée — ou **déprotéger tout de suite après usage**.

Les macros plus rares (§5.9.1) : `UNPROTECT_PTR(s)` « unprotects the object pointed to by the `SEXP s`, even if it is not the top item on the pointer protection stack » — « **dangerous to use in combination with `UNPROTECT` when the same object has been protected multiple times** », et « **superseded by multi-**

set based functions `R_PreserveInMSet` and `R_ReleaseFromMSet` ... *These functions should not be needed outside parsers.* » Et `PROTECT_WITH_INDEX` pour les objets qu'on remplace en cours de route.

● Concept 6 — Enregistrer les routines natives

DÉFINITION (§5.4).

« By "native" routine, we mean an entry point in compiled code. »

« In calls to `.C`, `.Call`, `.Fortran` and `.External`, R must locate the specified native routine by looking in the appropriate shared object/DLL. By default, R uses the operating-system-specific dynamic loader to lookup the symbol in all loaded DLLs ... Alternatively, the author of the DLL can explicitly register routines with R and use a single, platform-independent mechanism for finding the routines. »

Règle — les deux avantages (§5.4). « Registering routines has two main advantages : it provides a faster way to find the address of the entry point via tables stored in the DLL at compilation time, and it provides a run-time check that the entry point is called with the right number of arguments and, optionally, the right argument types. »

Le second avantage est le plus important : sans enregistrement, appeler une routine C avec le mauvais nombre d'arguments ne produit aucun message — seulement un comportement indéfini. C'est le même problème que l'argument `PACKAGE` du concept 2, traité à la racine.

MÉTHODE (§5.4).

« To register routines with R, one calls the C routine `R_registerRoutines`. This is typically done when the DLL is first loaded within the initialization routine `R_init_<dll name>`. `R_registerRoutines` takes 5 arguments. The first is the `DLLInfo` object passed by R to the initialization routine. ... The remaining 4 arguments are arrays describing the routines for each of the 4 different interfaces. »

Interface	Type de description
<code>.C</code>	<code>R_CMethodDef</code>
<code>.Call</code>	<code>R_CallMethodDef</code>
<code>.Fortran</code>	<code>R_FortranMethodDef</code>
<code>.External</code>	<code>R_ExternalMethodDef</code>

« Each argument is a `NULL` -terminated array ... [contenant] **the name of the routine by which it can be accessed in R, a pointer to the actual native symbol, and the number of arguments the routine expects to be passed from R** » :

```
SEXP myCall(SEXP a, SEXP b, SEXP c);

static const R_CallMethodDef callMethods[] = {
    {"myCall", (DL_FUNC) &myCall, 3},
    ...
};
```

Énoncé. Une fonction R est trop lente. Quel raisonnement mener, et quelle interface choisir le cas échéant ?

Étape 0 — *mesurer*. Avant tout, **profiler** (fiche 317) : `Rprof()` ... `summaryRprof()`, et lire la colonne `self`. Le §3.2 présentait le profilage comme le moyen d'« *identify ... pieces of R code that could benefit from being replaced by compiled code* » — **le profilage précède la décision**.

Étape 1 — *rester en R si possible*. « **First, consider working in interpreted R code ; if this is fast enough, this is normally the best option.** » (§5.9) La vectorisation (fiche 301) et la vision « objet entier » (fiche 308) résolvent beaucoup de cas — l'exemple des déterminants du §5.5 (fiche 305) en est la démonstration.

Étape 2 — *la question qui tranche entre `.C` et `.Call`*. « **You should also see if using `.C` is enough. If the task to be performed in C is simple enough involving only atomic vectors and requiring no call to R, `.C` suffices.** »

Deux critères : **uniquement des vecteurs atomiques**, et **aucun appel à R**. Si les deux sont remplis, `.C`.

Étape 3 — écrire l'interface `.C`. La fonction C est `void` — « ***the compiled code should not return anything except through its arguments*** ». Les types suivent la table du §5.2 : `double *` pour un `double`, `int *` pour un `integer` et pour un `logical`.

Étape 4 — les précautions de `.C`. **Nommer les arguments** (les noms deviennent ceux de la liste rendue), garder `NAOK = FALSE` sauf si le code C sait traiter `NA` / `NaN` / `Inf`, et **toujours passer** `PACKAGE` — « ***its use is highly desirable ... such name clashes are normally sufficient to cause R to crash*** ».

Étape 5 — si `.C` ne suffit pas. Listes, attributs, appels à R : c'est `.Call`. La signature devient `SEXP f(SEXP a, SEXP b)`, avec `R.h` et `Rinternals.h`.

Étape 6 — les deux responsabilités nouvelles. **Pas de copie** : « ***treat arguments you receive through these interfaces as read-only*** ». Et **la protection** : tout objet créé doit être `** PROTECT **`, avec un `UNPROTECT` qui équilibre.

Étape 7 — vérifier les types reçus. « ***Unless you are very sure about the type of the arguments, the code should check the data types*** » — `Rf_isReal`, `Rf_isInteger` ... puis **coercer** par `Rf_coerceVector` (**en protégeant**, puisqu'un objet est créé) ou **signaler** par `Rf_error`.

Étape 8 — enregistrer. `R_registerRoutines` dans `R_init_<dll>` : « ***a faster way to find the address ... and a run-time check that the entry point is called with the right number of arguments*** ».

Étape 9 — ce qu'on perd. Le débogueur de R **ne descend pas** dans le code compilé (fiche 317) : « ***no access to the statements in that language is provided ... A symbolic debugger such as gdb can be used*** ». **On échange de la vitesse contre de l'observabilité** — raison de plus pour ne descendre en C **qu'après** avoir mesuré.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« ma fonction R est trop lente »	profiler d'abord (fiche 317)
« faut-il passer en C ? »	« <i>if [R] is fast enough, this is normally the best option</i> »
« seulement des vecteurs, pas d'appel à R »	.C suffit
« manipuler des listes, des attributs »	.Call
« un nombre d'arguments variable »	.External — « <i>a little more general</i> »
« quel type C pour un logical R ? »	int * — comme integer
« du code venu de S-PLUS »	long * ne marche pas en 64 bits
« passer une chaîne à Fortran »	une seule , longueur fixe, déconseillé depuis 2019
« mes NA provoquent une erreur »	NAOK = TRUE — et les gérer côté C
« R plante à l'appel »	PACKAGE manquant — collision de symboles
« ma fonction C rend une valeur »	avec .C elle doit être void
« mon objet a été modifié chez l'appelant »	.Call ne copie pas
« stack imbalance in .Call »	PROTECT / UNPROTECT déséquilibrés
« mon objet disparaît en cours de route »	protection manquante après réassignation
« la pile de protection est pleine »	10 000 — ranger dans un objet, ou déprotéger tôt
« vérifier le type d'un argument »	Rf_isReal , Rf_isInteger , ...
« convertir un INTSXP en REALSXP »	Rf_coerceVector , en protégeant
« signaler une erreur depuis C »	Rf_error
« accélérer la recherche du symbole »	enregistrer les routines
« vérifier le nombre d'arguments »	enregistrement — contrôle à l'exécution
« déboguer mon code C »	gdb — pas le débogueur de R

Comment résoudre ce type d'exercice

Protocole « faut-il descendre en C ? » — 4 étapes.

1. **Profiler** — lire la colonne `self` (fiche 317).
2. Chercher une écriture **vectorisée** ou « objet entier » (fiches 301, 305, 308).
3. Si le C s'impose : **vecteurs atomiques seuls et aucun appel à R** → `.C` ; sinon `.Call` .
4. Accepter la contrepartie : **plus de responsabilités**, et **plus de débogueur R**.

Protocole « écrire une interface `.C` sûre » — 4 étapes.

1. Respecter **la table des types** — et se souvenir que `int` fait **32 bits partout**.
2. Fonction C `void` : le résultat sort **par les arguments**.
3. **Nommer les arguments** côté R : les noms deviennent ceux de la liste rendue.
4. **Toujours** `PACKAGE = "monpaquet"` , et laisser `NAOK = FALSE` sauf besoin explicite.

Protocole « écrire une interface `.Call` sûre » — 5 étapes.

1. Inclure `R.h` et `Rinternals.h` ; signature `SEXP f(SEXP ...)` .
2. **Vérifier les types** (`Rf_isReal` ...), puis **coercer** ou `Rf_error` .
3. `PROTECT` tout objet créé — y compris le résultat d'un `Rf_coerceVector` .
4. **Équilibrer** par `UNPROTECT(n)` ; ne jamais laisser la pile grossir sans raison.
5. Traiter les arguments reçus comme **en lecture seule**.

● Common mistakes

Erreur	Correction
Passer en C sans avoir profilé	« <i>if this is fast enough, this is normally the best option</i> »
Choisir <code>.Ca11</code> quand <code>.C</code> suffit	« <i>greater responsibilities</i> »
Utiliser <code>long *</code> pour un entier	<code>int</code> est 32 bits sur toutes les plateformes
Croire que « ça marche chez moi » suffit	« <i>it may appear to work on some</i> »
Retourner une valeur depuis une fonction <code>.C</code>	elle doit être <code>void</code>
Omettre <code>PACKAGE</code>	« <i>sufficient to cause R to crash</i> »
Laisser <code>NAOK = TRUE</code> sans gérer les <code>NA</code>	les valeurs spéciales passent non vérifiées
Passer plusieurs chaînes à Fortran	une seule , et déconseillé depuis 2019
Modifier un argument reçu par <code>.Ca11</code>	lecture seule — il n'est pas copié
Croire <code>PROTECT(p)</code> protéger <code>p</code> durablement	c'est l'objet, pas le pointeur
Oublier d'équilibrer <code>UNPROTECT</code>	« <i>stack imbalance in .CaLL</i> »
Tout protéger et tout déprotéger à la fin	la pile fait 10 000 places
Oublier de protéger le résultat d'une coercion	un nouvel objet est créé
Ne pas vérifier le type des arguments	« <i>the code should check the data types</i> »
Renvoyer un type interne (<code>LANGXP</code> , <code>PROMSXP</code> ...)	« <i>it is unsafe to do so</i> »
Accéder aux champs d'un <code>SEXPREC</code>	« <i>SEXP is an opaque pointer</i> »
Ne pas enregistrer ses routines	on perd la vitesse et le contrôle des arguments
Compter sur le débogueur de R dans le C	il faut <code>gdb</code>

Ultimate Review

L'ordre des choix. « *Before you decide to use `.Call` or `.External`, you should look at other alternatives. First, consider working in interpreted R code ; if this is fast enough, this is normally the best option. You should also see if using `.C` is enough. If the task to be performed in C is simple enough involving only atomic vectors and requiring no call to R, `.C` suffices. ... These interfaces allow much more control, but they also impose much greater responsibilities.* »

Le préfixe. « *Many of the functions ... have entry-point names with a `Rf_` prefix : if they are called from C code ... that prefix can be omitted. Users are encouraged to use the prefix when writing new C code.* »

Accès au système. `system`, `system2` ; « *not necessarily by a shell* » ; `shell` sous Windows ; `system.time` pour chronométrer — « *Timing on child processes is only available on Unix-alikes, and may not be reliable there* ».

`.C` et `.Fortran`. Premier argument : **le nom du symbole** (testable par `is.loaded`) · jusqu'à 65 arguments · **copiés à l'aller et au retour** dans une liste ; **les noms donnés deviennent ceux des composantes** · fonction C `void`, sous-programme Fortran `subroutine` · **table des types** :
`logical / integer` → `int *` / `INTEGER`, `double` → `double *` / `DOUBLE PRECISION`, `complex` → `Rcomplex *` / `DOUBLE COMPLEX`, `character` → `char **` / `CHARACTER(255)`, `raw` → `unsigned char *` / rien · `int` et `INTEGER` **font 32 bits partout** · logiques envoyés comme **0, 1** ou `INT_MIN` · `NAOK` : sinon `NA`, `NaN`, `Inf`, `-Inf` **provoquent une erreur** · `PACKAGE` — « *its use is highly desirable ... name clashes are normally sufficient to cause R to crash* » · `Csingle` pour passer des `float`.

`.Call` et `.External`. `.Call("f", a, b)` → `SEXP f(SEXP a, SEXP b)`, jusqu'à 65 objets, « *simpler to use* » · `.External("f", a, b)` → `SEXP f(SEXP args)`, `args` étant un `LISTSXP`, « *a little more general* » · en-têtes `R.h` et `Rinternals.h` · « *Neither `.Call` nor `.External` copy their arguments : you should treat arguments you receive through these interfaces as read-only.* »

`SEXP` . « *a pointer to a structure with `typedef SEXP` ... a variant type that can handle all the usual types of R objects* » — « *S imple EXP resion* » · « *SEXP is an opaque pointer, and the internals can only be accessed by the functions provided* » · un nœud : **en-tête `sxpinfo` de 64 bits + trois pointeurs** (attributs, précédent, suivant) ; **32 octets** en 32 bits, ~56 en 64 bits · **les cinq premiers bits** donnent le `SEXPTYPE`, parmi 32 possibles.

`SEXPTYPE` . `REALSXP` `INTSXP` `CPLXSXP` `LGLSXP` `STRSXP` (atomiques) · `VECSXP` = liste · `LISTSXP` = pairlist · `DOTSXP` · `NILSXP` · `SYMSXP` · `CLOSXP` · `ENVSXP` · internes : `LANGSXP`, `CHARSXP`, `PROMSXP` — « *it* »

is unsafe » d'en renvoyer · numéros **0-10 et 13-25, 11 et 12 retirés** (facteurs internes) et « *the gap cannot easily be reused* ».

Vérifier et convertir. « *Unless you are very sure about the type of the arguments, the code should check the data types* » — `Rf_isReal` , `Rf_isInteger` , `Rf_isString` , `Rf_isNull` , `Rf_isSymbol` , `Rf_isLogical` , `Rf_isComplex` , `Rf_isExpression` , `Rf_isEnvironment` · `Rf_error` pour signaler · `Rf_coerceVector` pour convertir — à **protéger**, un objet est créé · « *All the coercion functions do their own error-checking* ».

`PROTECT` . « *If you create an R object in your C code, you must tell R that you are using the object by using the `PROTECT` macro* » · « *it is the object which is protected, not the pointer variable ... a common mistake* » · « *Protecting an R object automatically protects all the R objects pointed to* » · pile **LIFO** : `UNPROTECT(n)` dépile les **n derniers** · « *The calls ... must balance ... R will warn about "stack imbalance in `.Call`"* » · **taille fixe, 10 000 par défaut** — « *not a good idea then to just `PROTECT` everything in sight* » ; les deux issues : **ranger dans un autre objet** ou **déprotéger aussitôt** · macros rares : `UNPROTECT_PTR` (**dangereuse**, supplantée par `R_PreserveInMSet` / `R_ReleaseFromMSet`) et `PROTECT_WITH_INDEX` .

Le raisonnement de la protection. « *we do not know (nor want to know) what is hiding behind the R macros and functions we use, and any of them might cause memory to be allocated, hence garbage collection ... It is usually wise to err on the side of caution.* »

Enregistrement. « *By "native" routine, we mean an entry point in compiled code.* » · deux avantages : « *a faster way to find the address of the entry point ... and a run-time check that the entry point is called with the right number of arguments and, optionally, the right argument types* » ·

`R_registerRoutines` dans `R_init_<dll>` , **cinq arguments** : le `DllInfo` , puis **quatre tableaux terminés par `NULL`** — `R_CMethodDef` , `R_CallMethodDef` , `R_FortranMethodDef` , `R_ExternalMethodDef` — chacun donnant **le nom, un pointeur vers le symbole, le nombre d'arguments**.

Active Recall

« *Before you decide to use `.Call` or `.External`, you should look at other alternatives. First, consider working in interpreted R code ; if this is fast enough, this is normally the best option. You should also see if using `.C` is enough. If the task to be performed in C is simple enough involving only atomic vectors and requiring no call to R, `.C` suffices. ... These interfaces allow much more control, but they also impose much greater responsibilities so need to be used with care.* » (§5.9)

Trois marches : R interprété — « *normally the best option* » —, puis `.C`, puis `.Call`.

Et une étape zéro, donnée par la fiche 317 : **profiler**. Le §3.2 présente le profilage comme le moyen d'« *identify ... pieces of R code that could benefit from being replaced by compiled code* ». Optimiser sans mesurer, c'est optimiser au hasard.

Mode R	C	Fortran
<code>logical</code>	<code>int *</code>	<code>INTEGER</code>
<code>integer</code>	<code>int *</code>	<code>INTEGER</code>
<code>double</code>	<code>double *</code>	<code>DOUBLE PRECISION</code>
<code>complex</code>	<code>Rcomplex *</code>	<code>DOUBLE COMPLEX</code>
<code>character</code>	<code>char **</code>	<code>CHARACTER(255)</code>
<code>raw</code>	<code>unsigned char *</code>	<i>aucun</i>

« *On all R platforms `int` and `INTEGER` are 32-bit. Code ported from S-PLUS (which uses `Long *` for logical and integer) will not work on all 64-bit platforms (although it may appear to work on some).* » (§5.2)

Noter que `logical` et `integer` **partagent** `int *` — c'est pourquoi les logiques circulent comme `0`, `1` ou `INT_MIN`.

« *Note that the compiled code should not return anything except through its arguments : C functions should be of type `void` and Fortran subprograms should be subroutines.* » (§5.2)

« *Normally these are copied before being passed in, and copied again to an R list object when the compiled code returns. If the arguments are given names, these are used as names for the components in the returned list object (but not passed to the compiled code).* »

Le cycle complet : R **copie** les arguments → le C **les modifie sur place** → R **recopie** le tout dans **une liste**. Nommer les arguments côté R rend cette liste lisible.

C'est l'exact opposé de `.Call`, où « *neither `.Call` nor `.External` copy their arguments* ».

« *Argument `PACKAGE` confines the search for the symbol name to a specific shared object (or use `"base"` for code compiled into R). Its use is highly desirable, as there is no way to avoid two package writers using the same symbol name, and such name clashes are normally sufficient to cause R to crash.* » (§5.2)

Il n'y a aucun espace de noms au niveau des symboles compilés — contrairement au code R (fiche 315). Deux DLL exportant `convolve` sont indiscernables pour le chargeur dynamique.

« *If it is not present and the call is from the body of a function defined in a package namespace, the shared object loaded by the first (if any) `useDynLib` directive will be used.* »

Et `R CMD check` **le vérifie** — point 10 de la fiche 319.

`.Call` : « *The C side of the interface is `SEXP convolve2(SEXP a, SEXP b)`* » — **un paramètre par objet**, jusqu'à **65**.

`.External` : « *the C side of the interface is different, **having only one argument** : `SEXP` `convoLveE(SEXP args)` . Here `args` is a `LISTSXP` , a Lisp-style pairlist from which the arguments can be extracted.* » (§5.9)

« *Generally the `.Call` interface is simpler to use, but `.External` is a little more general.* »

`.External` accepte un nombre quelconque d'arguments — il faut les extraire de la pairlist, ce qui est plus lourd mais plus souple.

Les deux exigent `R.h` et `Rinternals.h` , et aucun des deux ne copie ses arguments.

« *All the R objects you will deal with will be handled with the type `SEXP` , which is a pointer to a structure with `typedef SEXP` . Think of this structure as a variant type that can handle all the usual types of R objects.* » (§5.9) — « *S imple EXP ressession* ».

« *Think rather of a model such as that used by Visual Basic, in which R objects are handed around in C code (as they are in interpreted R code) as the variant type, and the appropriate part is extracted ... only when it is needed. As in interpreted R code, much use is made of coercion.* »

Et *R Internals* §1.1 précise : « *the internal structure of the `SEXP` is not made available to R Extensions : rather `SEXP` is an opaque pointer, and the internals can only be accessed by the functions provided* ».

On ne lit pas les champs d'un `SEXP` : on passe par les macros et fonctions publiques de `Rinternals.h` .

SEXPTYPE	R
REALSXP INTSXP CPLXSXP LGLSXP STRSXP	les vecteurs atomiques
VECSXP	une liste
LISTSXP	une pairlist
SYMSXP · CLOSXP · ENVSXP · NILSXP · DOTSXP	symbole, fonction, environnement, NULL , ...

C'est la table de `typeof()` (fiche 303), vue du côté C.

R Internals §1.1.1 : « Values 11 and 12 were used for internal factors and ordered factors and have since been withdrawn. Note that the `SEXPTYPE` numbers are stored in saved objects and that the ordering of the types is used, so the gap cannot easily be reused. »

Un trou dans la numérotation, gravé dans le format des fichiers sauvegardés — on ne peut pas le combler sans casser la lecture des anciens `.RData`.

Et « *although it is possible to return objects of internal types, it is unsafe to do so* ».

« *If you create an R object in your C code, you must tell R that you are using the object by using the `PROTECT` macro on a pointer to the object. This tells R that the object is in use so it is not destroyed during garbage collection.* » (§5.9.1)

L'erreur nommée : « *Notice that it is the object which is protected, not the pointer variable. It is a common mistake to believe that if you invoked `PROTECT(p)` at some point then `p` is protected from then on, but that is not true once a new object is assigned to `p`.* »

`PROTECT(p)` met à l'abri **ce que `p` désigne à cet instant**. Réassigner `p` laisse le **nouvel** objet sans protection — et l'ancien protégé pour rien.

Et la protection est transitive : « *Protecting an R object automatically protects all the R objects pointed to in the corresponding `SEXPREC` — for example all elements of a*

protected list are automatically protected. »

Le manuel pose lui-même la question (§5.9.1) : « *the reader may ask **how the R object could possibly get removed during those manipulations**, as it is just our C code that is running.* »

Et il répond : « *As it happens, we can do without the protection in this example, but in general we do not know (nor want to know) what is hiding behind the R macros and functions we use, and any of them might cause memory to be allocated, hence garbage collection and hence our object `ab` to be removed. It is usually wise to err on the side of caution and assume that any of the R macros and functions might remove the object.* »

C'est une règle de discipline, pas de nécessité locale. Une macro de `Rinternals.h` peut allouer ; allouer peut déclencher le ramasse-miettes ; le ramasse-miettes peut reprendre tout objet non protégé. **On ne peut pas auditer chaque macro** — donc on protège.

« *The programmer is solely responsible for housekeeping the calls to `PROTECT`. ... The protection mechanism is stack-based, so `UNPROTECT(n)` unprotects the last `n` objects which were protected. The calls to `PROTECT` and `UNPROTECT` must balance when the user's code returns and should balance in all functions. R will warn about "stack imbalance in `.Call`" (or `.External`) if the housekeeping is wrong.* » (§5.9.1)

Et la pile est finie : « *The pointer protection stack has a fixed size (default 10,000) and can become full. It is not a good idea then to just `PROTECT` everything in sight and `UNPROTECT` several thousand objects at the end.* »

Les deux issues données par le manuel : « *It will almost invariably be possible to either assign the objects as part of another object (which automatically protects them) or unprotect them immediately after use.* »

*« **Unless you are very sure about the type of the arguments, the code should check the data types. You can use functions like `Rf_isReal`, `Rf_isInteger` and `Rf_isString`** ... Other such functions include `Rf_isNull`, `Rf_isSymbol`, `Rf_isLogical`, `Rf_isComplex`,

`Rf_isExpression` , and `Rf_isEnvironment` . All of these take a `SEXP` as argument and return 1 or 0. »* (§5.9.3)

« *Sometimes you have no other option except to generate an error. You can use the function `Rf_error` for this. It is usually better to coerce the object to the correct type.* »

```
newSexp = PROTECT(Rf_coerceVector(oldSexp, REALSXP));
```

« *Protection is needed as a new object is created ; the object formerly pointed to by the `SEXP` is still protected but now unused.* »

« *All the coercion functions do their own error-checking, and generate `NA` s with a warning or stop with an error as appropriate.* »

« *Registering routines has two main advantages : it provides a faster way to find the address of the entry point via tables stored in the DLL at compilation time, and it provides a run-time check that the entry point is called with the right number of arguments and, optionally, the right argument types.* » (§5.4)

Le second est le plus important. Sans enregistrement, appeler une routine C avec le mauvais nombre d'arguments ne produit **aucun message** — seulement un comportement indéfini, souvent un plantage plus loin.

Comment : « *one calls the C routine `R_registerRoutines` ... within the initialization routine `R_init_<dll name>` » , avec **cinq arguments** — le `DllInfo` , puis quatre tableaux terminés par `NULL` : `R_CMethodDef` , `R_CallMethodDef` , `R_FortranMethodDef` , `R_ExternalMethodDef` .*

Chaque entrée donne « *the name of the routine by which it can be accessed in R, a pointer to the actual native symbol, and the number of arguments* ».

Trois choses, réparties dans les fiches.

1. **L'observabilité** — fiche 317, §9 : « *The debugger provides access only to interpreted expressions. If a function calls a foreign language (such as C) then no access to the*

statements in that language is provided. A symbolic debugger such as `gdb` can be used to debug compiled code. »

2. **La sécurité de la sémantique de copie** — §5.9 : « *Neither `.Call` nor `.External` copy their arguments : you should treat arguments you receive through these interfaces as read-only.* » Une modification silencieuse touche l'objet de l'appelant.
3. **La gestion automatique de la mémoire** — §5.9.1 : « *The programmer is solely responsible for housekeeping the calls to `PROTECT`.* »

Le manuel résume : « *These interfaces allow much more control, but they also impose much greater responsibilities so need to be used with care.* »

D'où la règle : ne descendre en C **qu'après avoir profilé** et constaté que R ne suffit pas.

Flashcards

Question	Réponse
Les quatre interfaces vers le code compilé ?	<code>.C</code> · <code>.Fortran</code> · <code>.Call</code> · <code>.External</code>
Que conseille le manuel avant tout ?	Rester en R interprété si c'est assez rapide
Et avant cela ?	Profiler (fiche 317)
Quand <code>.C</code> suffit-il ?	Vecteurs atomiques seuls et aucun appel à R
Que signifie le préfixe <code>Rf_</code> ?	Le nom du point d'entrée ; omissible en C , recommandé
Les fonctions d'accès au système ?	<code>system</code> et <code>system2</code>
Que peut-on supposer du premier argument ?	Qu'il est passé pour exécution , pas forcément par un shell
Sous Windows, pour un shell ?	<code>shell</code>
La fonction de chronométrage ?	<code>system.time</code>
Est-elle fiable sur les processus fils ?	Unix seulement , et « <i>may not be reliable</i> »
Premier argument de <code>.C</code> ?	Le nom du symbole , en chaîne
Comment tester qu'il est chargé ?	<code>is.loaded("...")</code>
Combien d'arguments au maximum ?	65
Les arguments sont-ils copiés avec <code>.C</code> ?	Oui — à l'aller et au retour
Que deviennent les noms d'arguments ?	Ceux des composantes de la liste rendue
Type C d'un <code>logical</code> R ?	<code>int *</code>
D'un <code>integer</code> ?	<code>int *</code> aussi
D'un <code>double</code> ?	<code>double *</code>
D'un <code>complex</code> ?	<code>Rcomplex *</code>
D'un <code>raw</code> ?	<code>unsigned char *</code> — rien en Fortran

Question	Réponse
Combien de bits fait <code>int</code> ?	32 , sur toutes les plateformes
Le piège du code venu de S-PLUS ?	Il utilise <code>long *</code>
Où est défini <code>Rcomplex</code> ?	<code>R_ext/Complex.h</code>
Combien de chaînes peut-on passer à Fortran ?	Une seule , de longueur fixe
Depuis quand est-ce déconseillé ?	2019
Comment une valeur logique circule-t-elle ?	0, 1 , ou <code>INT_MIN</code> pour <code>NA</code>
Que fait <code>NAOK = FALSE</code> ?	Il rejette <code>NA</code> , <code>NaN</code> , <code>Inf</code> , <code>-Inf</code>
Que fait <code>Csingle</code> ?	Il permet de passer des <code>float</code> / <code>REAL</code>
À quoi sert <code>PACKAGE</code> ?	À restreindre la recherche du symbole
Que dit le manuel de son usage ?	« <i>highly desirable</i> »
Que risque-t-on sans lui ?	Un plantage de R
Que se passe-t-il s'il est absent dans un paquet ?	Le premier <code>useDynLib</code> est utilisé
Comment une fonction <code>.C</code> rend-elle son résultat ?	Par ses arguments — elle est <code>void</code>
La signature C de <code>.Call</code> ?	<code>SEXP f(SEXP a, SEXP b)</code>
Celle de <code>.External</code> ?	<code>SEXP f(SEXP args)</code>
Qu'est-ce que <code>args</code> ?	Un <code>LISTSXP</code> — une pairlist
Laquelle est plus simple ?	<code>.Call</code>
Laquelle est plus générale ?	<code>.External</code>
Les deux en-têtes à inclure ?	<code>R.h</code> et <code>Rinternals.h</code>
<code>.Call</code> copie-t-il ses arguments ?	Non
Comment doit-on les traiter ?	En lecture seule

Question	Réponse
Qu'est-ce qu'un <code>SEXP</code> ?	Un pointeur vers un <code>SEXPREC</code>
Que signifie l'acronyme ?	Simple EX pression
Peut-on lire les champs d'un <code>SEXPREC</code> ?	Non — pointeur opaque
De quoi est fait un nœud ?	Un en-tête <code>sxpinfo</code> + trois pointeurs
Sa taille en 64 bits ?	Environ 56 octets
Combien de bits pour le type ?	Les cinq premiers de <code>sxpinfo</code>
Combien de <code>SEXPTYPE</code> possibles ?	32
Le <code>SEXPTYPE</code> d'une liste ?	<code>VECSXP</code>
Celui d'une pairlist ?	<code>LISTSXP</code>
Celui d'une fonction ?	<code>CLOSXP</code>
Celui d'un environnement ?	<code>ENVSXP</code>
Celui d'un <code>numeric</code> ?	<code>REALSXP</code>
Celui d'un symbole ?	<code>SYMSXP</code>
Trois types internes ?	<code>LANGSXP</code> · <code>CHARSXP</code> · <code>PROMSXP</code>
Peut-on en renvoyer ?	« <i>it is unsafe to do so</i> »
Quels numéros sont retirés ?	11 et 12 — anciens facteurs internes
Pourquoi ne peut-on les réutiliser ?	Ils sont stockés dans les objets sauvegardés
Trois fonctions de test de type ?	<code>Rf_isReal</code> · <code>Rf_isInteger</code> · <code>Rf_isString</code>
Que rendent-elles ?	1 ou 0
Comment signaler une erreur depuis C ?	<code>Rf_error</code>
Comment convertir un objet ?	<code>Rf_coerceVector</code>
Faut-il le protéger ?	Oui — un objet est créé

Question	Réponse
Que fait <code>PROTECT</code> ?	Il met l'objet à l'abri du ramasse-miettes
Protège-t-il le pointeur ?	Non — « <i>a common mistake</i> »
La protection est-elle transitive ?	Oui — les éléments d'une liste protégée le sont
Quel est le mécanisme de la pile ?	LIFO — <code>UNPROTECT(n)</code> dépile les n derniers
Que se passe-t-il si le compte est faux ?	« <i>stack imbalance in</i> <code>.CALL</code> »
Quelle est la taille de la pile ?	10 000 par défaut
Les deux façons d'éviter de la saturer ?	Ranger dans un autre objet · déprotéger tôt
Que fait <code>UNPROTECT_PTR</code> ?	Il déprotège un objet même hors du sommet
Est-ce recommandé ?	Dangereux — supplanté par les multi-ensembles
La macro pour un objet remplacé en route ?	<code>PROTECT_WITH_INDEX</code>
Qu'est-ce qu'une routine « native » ?	Un point d'entrée dans du code compilé
Comment R la trouve-t-il par défaut ?	Par le chargeur dynamique du système
Les deux avantages de l'enregistrement ?	Plus rapide · vérification des arguments
Quelle fonction C l'effectue ?	<code>R_registerRoutines</code>
Où l'appeler ?	Dans <code>R_init_<nom de la dll></code>
Combien d'arguments prend-elle ?	Cinq
Quel est le premier ?	L'objet <code>DllInfo</code>
Les quatre types de description ?	<code>R_CMethodDef</code> · <code>R_CallMethodDef</code> · <code>R_FortranMethodDef</code> · <code>R_ExternalMethodDef</code>
Que contient chaque entrée ?	Le nom, le pointeur, le nombre d'arguments
Comment se terminent ces tableaux ?	Par <code>NULL</code>
Peut-on déboguer le C avec les outils de R ?	Non — il faut <code>gdb</code>